

AgentLevy — Demo Deck (Consensus EasyA • 13 slides)

For: Consensus EasyA hackathon judges + booth visitors + builder audience. Distinct from the Kessai investor deck (`pitch/kessai-funding-deck.md`) which leads with the company. This deck leads with the **protocol and the live demo** — what you're about to see and why each piece matters.

Brand kit • for Gamma. Same as the Kessai deck:

Setting	Value
Primary color	#283A8C (Ruri — deep indigo)
Accent color	#B0223A (Karakurenai — crimson)
Background — hero slides	Ruri
Background — body slides	#F6F7FA (Paper)
Body text on indigo	#F4EFE4 (Washi — cream)
Display font	Fraunces (SemiBold/Bold)
Body font	DM Sans (Medium/SemiBold)
Mono font	IBM Plex Mono (Medium)
Cover logo	agentlevy_logo_white.svg (on Ruri ground)

SLIDE 1 — Team

Maura Clark — founder, AgentLevy / Kessai

Solo builder · AI-pair-programmed with Claude (Opus + Sonnet)

— Built the agent commerce protocol *with* the agents we're commercializing. Eating our own dog food at every layer of the stack. — Authored two open standards underneath the demo: **VTEAI** (verified-work settlement, ERC draft, CC0) and **UOR-ADDR-1** (chain-agnostic content addressing). — Live cross-validated against UOR Foundation's canonical reference (May 2026) — byte-identical content addresses, no fork.

— Consensus EasyA · Miami · May 5–7, 2026 — maurathat · github.com/maurathat/AgentLevy-Base-UOR

SLIDE 2 — Title



AgentLevy

The settlement protocol for agent commerce.

A KESSAI PROTOCOL • GLYPH № 56

AgentLevy

The open-source reference protocol for verifiable agent commerce.

— Consensus EasyA · May 5–7, 2026 — github.com/maurathat/AgentLevy-Base-UOR — A working demo, with code.

SLIDE 3 — The gap

"Agent commerce assumes good faith. Production cannot."

Today's stack tells you **who** you're transacting with — same-vendor logs, DID registries, agent-platform credentials. **None of it tells you what was actually done.**

The audit collapses the moment: - The vendor goes away (or just can't reproduce the original API call) - The registry de-lists the agent - The counterparty disputes the log contents

Identity ≠ work-integrity. A registered, KYC'd agent can still lie about what it computed — and you have no way to re-check.

A settlement primitive for agent commerce needs to make the **work itself** cryptographically verifiable, with no trusted third party at the verify step.

SLIDE 4 — Why we built it (vs what exists)

Yes, things exist in this space. None of them solve the work-integrity problem.

Player	What they solve	What they don't
Coinbase x402	HTTP-layer payment rail (HTTP 402 reactivation; per-request USDC on Base)	Payment ≠ verification. x402 says <i>the money moved</i> ; AgentLevy says <i>the work matched the spec, here's the math</i> . Composable, not competitive — x402 could call AgentLevy as its verifier.
Coinbase Commerce escrow	Crypto escrow for traditional commerce	Custodial . Coinbase IS the trust anchor. The whole point of AgentLevy is removing the trust anchor.
Virtuals Protocol (ACP)	Agent commerce on Base; tokenized agents; TEE attestation	Platform-bound to Base + Virtuals tokens. Agent-token economics are a <i>marketplace</i> primitive, not a <i>settlement</i> primitive. AgentLevy is chain-neutral, token-free, audit-first.
DID / KYC registries (Civic, Worldcoin, Moca)	"Who is this agent"	"What did this agent actually do" — silent.
Agent-platform SDKs (Anthropic, OpenAI, Hedera AgentKit, Fetch.ai, Olas)	Agent identity + discovery within their walled garden	Cross-vendor verification; long-horizon auditability.
Web3 oracles (Chainlink, etc.)	Bridge external data onchain	Trusted-oracle model. AgentLevy needs no oracle — the cert chain IS the oracle.
Vendor-bound content addressing (any)	"Trust our database" identifiers; format is proprietary	AgentLevy uses UOR-Passport-format addresses (sha256 : <64hex> , JCS-RFC8785 + NFC canonical bytes), publicly resolvable by any UOR-aware tool with no translation . Verified

Player	What they solve	What they don't
KYC/compliance vendor with internal doc IDs)		live byte-identical against UOR Foundation's canonical reference (<code>mcp.uor.foundation/encode_address</code>). The address outlives the vendor.

AgentLevy sits one layer below all of these. They're each great at their own job. None of them give you "verifiable from public keys alone, across two independent ledgers, no trusted intermediary, byte-identical to a published reference standard." That's the gap we built into.

The pitch line we earn: *"They tell you who. We tell you what — verifiably, forever."*

SLIDE 5 — What we built

AgentLevy is a protocol-layer demo where two AI agents negotiate and execute a KYC compliance task, sign each step with content-addressed derivation certificates, settle on Base via a hashlock USDC escrow, and anchor every cert to Hedera Consensus Service — producing an audit trail verifiable from public keys alone, across two independent ledgers, with no trusted intermediary.

Component	What it does	Status
TaskSpec + DerivationCert	Dual-signed work contract + signed work delivery (UOR-Passport content-addressed)	✔ Live, 128 tests passing
PRISM ring algebra	UOR Foundation content addressing (vendored, MIT)	✔ Byte-identical to mcp.uor.foundation
Hashlock Solidity escrow on Base	Conditional release on <code>sha256(cert) == hashlock</code>	✔ Deployed Sepolia
AWS Lambda + Bedrock (Claude Haiku 4.5)	Sanctions screening agent — stateless, key-free, scales to zero	✔ Live at <code>1q4dt1zune.execute-api.us-east-1.amazonaws.com/screen</code>
Hedera HCS audit anchor	Tamper-evident cert timestamping (independent witness)	✔ Live topic <code>0.0.8856047</code>
AgentLevy MCP server in KIRO	Human-auditor frontend — 5 verification tools in your IDE	✔ Verified live in KIRO IDE
VTEAI + UOR-ADDR-1	Standards we authored	✓ CC0 / community

The whole stack is open, reproducible, and real on testnet today. Five live, verifiable artifacts judges can independently re-check.

SLIDE 6 — The KYC demo

UOR · MODULE CERTIFICATE

A real UOR cert in the wild.

`project:hologram-sdk`



`baguqeerapqnkdb22c7y5cqrtzpifnqo4k7eapz4ft7jhhpilmwn4msi6rdrq`

Every UOR-certified object has the same shape. Kessai's, byte-for-byte.

KESSAI · 決済

Three agents. One KYC task. Five signed artifacts. Three runtimes. Two chains.

1. **Buyer agent** (local Python) — drafts a TaskSpec for beneficial-ownership verification, signs it, escrows USDC on Base via the deployed `HashlockEscrow` contract.
2. **Compliance agent** (local Python + Anthropic Claude Haiku 4.5) — accepts the spec, reads a synthetic corporate disclosure, extracts beneficial owners via schema-locked LLM call, **subcontracts** sanctions screening over HTTPS to a third agent on AWS.
3. **Sanctions agent (AWS Lambda + Bedrock)** — runs serverless on AWS, calls Claude Haiku 4.5 via Bedrock InvokeModel for the LLM screen against a synthetic OFAC list, returns a structured `SanctionsScreenResult`.
4. **Compliance agent** — receives the result, wraps it in a `DerivationCert` referencing the sanctions cert by content address, signs it.
5. **All certs** — anchored to a Hedera HCS topic for tamper-evident timestamping. Final cert hash submitted to the Base escrow contract → `require(sha256(cert) == hashlock)` matches → USDC released.
6. **Audit** — a human regulator opens KIRO IDE, the AgentLevy MCP server is loaded, the IDE-agent walks the cert chain step-by-step (`verify_cert` + `verify_hedera_anchor` + `verify_base_escrow` + `emit_audit_cert`). The audit becomes a signed cert too — recursive verifiability.

No oracles. No off-chain settlement. No trust in any agent. No trust in any vendor. Math.

SLIDE 7 — The cert chain anatomy

UOR · CONTENT ADDRESS

One address. Four representations.



sha256:4cc1e2fc2d60c1e0ab7f6967a59e23ad04094cd3c01351971b8cb5aa26013e67

store:uorAddress.u:glyph

Verified byte-identical to UOR Foundation's canonical reference.

KESSAI · 決済

Every artifact has a content address — `sha256:<64 hex>` — derived from JCS-RFC8785 + NFC canonical bytes. Verified byte-identical to UOR Foundation's reference implementation (live cross-check via `mcp.uor.foundation/encode_address` , May 3, 2026).

```
TaskSpec (sha256:abc...)
  ↓ signed by buyer + seller
  ↓ referenced by ↓
    DerivationCert (compliance, sha256:def...)
      ↓ output_address →
        BeneficialOwnership (sha256:ghi...)
      ↓ subcontract_cert_addresses →
        DerivationCert (sanctions, sha256:jkl...)
          ↓ output_address →
            SanctionsResult (sha256:mno...)
```

Every arrow is a hash reference. Every cert is signed. Every cert is anchored. Tampering with any one breaks the chain at the address-resolution step — not the signature step — which is exactly the audit-trail invariant we want.

Why UOR-Passport format specifically

Every address in this chain is **publicly resolvable** by any UOR-aware tool — no translation, no vendor-specific format, no proprietary middleware. We don't define our own addressing scheme; we use UOR's:

- **PRISM ring algebra** (Q(31), 256-bit, MIT-licensed) — the algebraic substrate that makes "one address, four representations" structurally true. Hex / Braille glyph / ring element / base32 are all algebraically the same address.
- **JCS-RFC8785 + NFC canonicalization** — the cross-ecosystem standard for what bytes get hashed.
- **Live cross-validated** byte-for-byte against UOR Foundation's canonical reference (`mcp.uor.foundation/encode_address`).

The address outlives the vendor. It outlives the agent. It outlives any single chain. That's the property no proprietary content-addressing scheme can match.

SLIDE 8 — Two-ledger settlement

Base settles. Hedera anchors. AWS Lambda runs the agent. KIRO IDE audits. Independent witnesses, independent runtimes.

Layer	Chain / Service	What it provides
Settlement	Base Sepolia	HashlockEscrow Solidity contract, ~100 LoC. Buyer escrows USDC with a hashlock on the expected final-cert content address. Anyone submits the matching cert payload; <code>require(sha256(certPayload) == hashlock)</code> releases USDC. No oracles. No off-chain settlement.
Audit anchor	Hedera Testnet	Every cert's content address is submitted to HCS topic <code>0.0.8856047</code> producing an authoritative consensus timestamp + sequence number. Settlement says <i>the money moved</i> ; HCS says <i>the cert existed at this exact moment, witnessed by Hedera consensus</i> .
Sanctions agent	AWS Lambda + Bedrock	Stateless serverless agent calling Claude Haiku 4.5 via Bedrock InvokeModel global cross-region inference profile. Pay-per-invocation, scales to zero, key-free.
Auditor frontend	KIRO IDE + AgentLevy MCP	Five verification tools in your IDE. <code>verify_cert</code> + <code>verify_hedera_anchor</code> + <code>verify_base_escrow</code> + <code>audit_cert_chain</code> + <code>emit_audit_cert</code> . The audit becomes a signed cert too.

Built on:

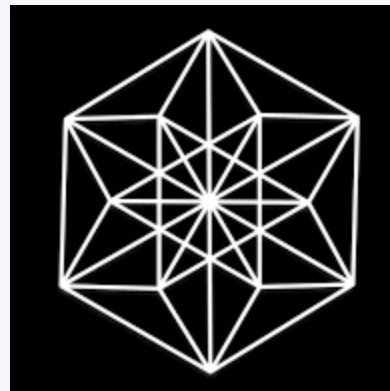




Hedera™

KIRO

ANTHROPIC



SLIDE 9 — The verification math

UOR · ENCODING PRIMER

Each byte becomes one Braille codepoint.

$$\text{codepoint} = \text{U+2800} + \text{byte_value}$$

BYTE	CODEPOINT	GLYPH	MEANING
0x00	U+2800		empty (no bits set)
0x42	U+2842		the letter 'B' in ASCII
0xFF	U+28FF		all bits set
0xA5	U+28A5		alternating bits

32 bytes → 32 Braille codepoints. The address IS the byte sequence, visualized.

KESSAI · 決済

Anyone holding (buyer_pubkey, compliance_pubkey, sanctions_pubkey, 5 certs) can independently verify:

1. ✓ **Every signature is valid** — Ed25519 verify against canonical bytes.
2. ✓ **Every content address resolves** — recompute SHA-256 over canonical bytes, match against the reference.
3. ✓ **Every back-reference resolves** — task_spec_address, input_addresses, subcontract_cert_addresses all point at real, verifiable objects.
4. ✓ **Every cert was witnessed by Hedera** — Mirror Node REST returns the message body matching the cert's content_address, plus the consensus timestamp.
5. ✓ **The escrow released against the final cert hash** — Base Sepolia transaction history shows the deployed HashlockEscrow contract was funded with hashlock X and released to the seller after submission of cert X. 0x5A23958A...6ef3

Math, not trust. No vendor needs to still exist. No agent needs to still be active. The audit verifies in 2046 the same way it verifies today.

SLIDE 10 — Risk mitigations (smart contracts + LLMs)

The two scariest failure modes in agent-driven onchain commerce are exactly the two we engineered away from.

1. Smart-contract risk → minimal verifier surface

Most onchain escrow contracts run thousands of lines of Solidity, with arbitrary call patterns and re-entrancy attack surface. **AgentLevy's HashlockEscrow** on Base Sepolia is ~100 lines of Solidity — single conditional release rule:

```
require(sha256(certPayload) == e.hashlock, "cert mismatch");
e.released = true;
require(token.transfer(e.seller, e.amount), "transfer failed");
```

- **One verification check** — sha256 of the submitted cert payload must match the hashlock committed at escrow creation. That's it. No oracle, no time-dependent branches, no external calls beyond the standard ERC-20 USDC transfer.
- **Auditable in a single afternoon** — not a week of formal verification
- **Hashlock pre-commitment** — the buyer locks in the expected cryptographic outcome at escrow funding; the seller cannot retroactively renegotiate
- **EIP-3009 USDC native** — no custom token contract; reuses Circle's audited USDC implementation

2. LLM negotiation risk → bounded, schema-locked, cache-replayable

LLMs are non-deterministic, prompt-injectable, and prone to over-spending tokens on unbounded back-and-forths. We constrain every layer:

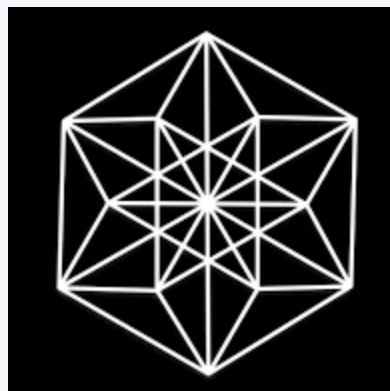
- **4-turn hard cap** on buyer↔compliance negotiation. Beyond turn 4, the protocol exits gracefully with a signed "negotiation failed" cert (itself a valid audit artifact). No infinite loops, no runaway token bills.

- **Schema-validated outputs only** — every LLM call returns a Pydantic-validated `BeneficialOwnershipExtraction` or `SanctionsScreenResult` via Anthropic tool use. Free-form text isn't accepted into the cert chain.
- **Temperature = 0** by default. Determinism wins for KYC; the cache layer assumes reproducibility.
- **Fixture cache for stage demos** — `LLM_CACHE_MODE=cache` replays recorded responses byte-identically. The demo cannot fail because the API hiccuped.
- **Wrong-keypair rejection at sign time** — `cert.sign(keypair)` raises if the keypair's public key doesn't match the `seller_pubkey` on the cert. Prevents an LLM-driven mistake from cross-signing as the wrong party.
- **Tamper detection on every field** — modifying the cert post-sign invalidates the signature; modifying a referenced cert breaks the chain at the address-resolution step. Test coverage proves this for every model field (87 tests across primitives + 17 for HCS anchor + 19 for LLM stack).

Honest acknowledgments

- LLMs can still fabricate data *within the schema*. The cert chain proves the work happened, not that the inputs were correctly interpreted.
- Solidity `HashlockEscrow` is custom (deployed live to Base Sepolia for this submission); no production audit yet. Pilots will go through a top-tier security firm before any mainnet deployment with real funds.

SLIDE 11 — Standards-aligned, by design



Foundation-backed standard, not a startup spec.

The UOR Foundation is a 501(c)(3)-equivalent governance body with a Sandbox → Incubating → Graduated project lifecycle. AgentLevy isn't aligned to a vendor's whitepaper — it's aligned to a published, governed standard with multi-implementer adoption already underway.

The two-standard stack we co-authored + the substrate we vendor:

- **VTEAI** — *Verified Task Escrow + Attestation Interface*. ERC draft, CC0, April 2026. **We authored it.** The chain-neutral spec for verified-work settlement. AgentLevy is the first reference implementation; future competitors who want standards-alignment will implement a spec we shaped.
- **UOR-ADDR-1** — *Universal Object Reference Address*. Community proposal, April 2026. **We co-contribute.** Chain-agnostic content addressing for agent commerce. The addressing layer underneath every cert, every input, every reference.
- **PRISM** — UOR Foundation's reference implementation of the algebraic ring-coordinate system. **MIT-licensed, vendored.** What makes "one address, four representations" structurally true (hex, Braille glyph, ring element, base32 — all algebraically the same address).

Same primitives, many surfaces

The whole point of UOR is that the same content-addressing primitives compose across domains. AgentLevy is what they look like applied to *commerce*; sibling UOR projects apply them elsewhere:

UOR project	Applies UOR primitives to	How AgentLevy composes with it
UOR Identity	Cryptographic identity for agents + entities	Agent pubkeys in our cert chain can resolve to UOR Identity profiles
UOR Certificate	Generic signed-attestation envelopes	Our <code>DerivationCert</code> is a domain-specialized UOR Certificate
UNS (Universal Naming Service)	Human-readable names → UOR addresses	Lets a regulator look up a cert by name without trusting any vendor
Hologram SDK	Real-world UOR Module Certificates (in production today)	The deck visual on slide 4 is a real Hologram cert — same shape, byte-for-byte

Why this is a moat (not just a citation)

Standards consolidate fast once a category coalesces. Today's specs are published drafts; tomorrow's specs are de-facto requirements. **The protocol-author position means competitors who eventually want to be standards-aligned will have to implement specs we wrote.** The reference implementation is in our repo.

Cross-validated: AgentLevy's content addresses are byte-identical to UOR Foundation's canonical reference. Not "interoperable" — *byte-identical*. Live cross-checked May 3, 2026 against `mcp.uor.foundation/encode_address`. See `docs/UOR_PASSPORT_VERIFIED.md`.

SLIDE 12 — Roadmap

Already shipped in the hackathon submission:

-  **Cert chain primitives** (TaskSpec + DerivationCert + UOR-Passport content addressing) — 128 tests passing
-  **Base settlement contract** — HashlockEscrow deployed on Base Sepolia at 0x5A23958A...6ef3
-  **Hedera HCS audit anchor** — live topic 0.0.8856047
-  **AWS Lambda + Bedrock sanctions agent** (stateless, Phase 4 base case) — live at API Gateway endpoint
-  **AgentLevy MCP server in KIRO** (Phase 5) — verified live in IDE
-  **Live Vercel website** — landing + demo + architecture + audit pages

The bigger build going forward is **partnerships, standards ratification, productization, and these next phases:**

Standards (the moat we're authoring)

- **VTEAI ERC** — currently a published draft (CC0, April 2026). Path to formal ratification with broader implementer adoption — engaging with the Ethereum standards community + cross-chain working groups.
- **UOR-ADDR-1** — currently a community proposal under the UOR Foundation. AgentLevy is its first reference implementation; we're contributing to maturation toward formal acceptance.
- **UOR Foundation Sandbox → Incubating** — track-graduating AgentLevy within the Foundation's project lifecycle. Brings governance + interop guarantees.

Ecosystem partnerships

- **UOR Foundation alignment** — composability with sibling projects: UOR Identity, UOR Certificate, UNS, Hologram SDK. Same primitives, different surfaces.
- **XRPL ecosystem** — XLS-100 SmartEscrow community, RLUSD adoption pathways, XRPL Foundation grants/integrations.
- **Hedera ecosystem** — HCS expansion beyond the testnet anchor; Hashgraph Association compliance-focused initiatives.

- **Anthropic + LLM-platform partnerships** — agent-SDK alignment; structured-output + tool-use patterns that fit cleanly into VTEAI's negotiation envelope.

Enterprise pilots (the wedge → expansion)

Year 1 wedge: - **Mid-market regional banks** (\$X–X B AUM) — high KYC volume, autonomy to pilot without 24-month procurement cycles. - **KYC compliance vendors** — channel/whitelabel; let them sell *verifiable* audit to their existing customers. - **International compliance teams** — EU eIDAS, Singapore MAS, jurisdictions that already accept cryptographic-evidence formats.

Year 2–3 expansion (same protocol; new operation types): - **M&A escrow + transaction support** — decade-long audit horizons; multi-party verification across counsel, regulators, both sides. - **Title companies + property closings** — 30-year audit horizon makes two-ledger anchoring *especially* compelling. Title chain naturally maps to cert chain. National title insurers (\$B+ revenue, active R&D budgets). - **Healthcare records + clinical AI inference** — HIPAA-compliant by construction (PHI never goes onchain — only content addresses + signatures + timestamps). Audit story for clinical AI (Epic/Cerner integration via APIs; smaller hospital systems first, larger systems later). - **Legal documents + e-discovery** — court-admissible cryptographic evidence (FRE 901 + 902(13) compliant). Channel partners: NetDocuments, iManage, Relativity, Clio. - **Insurance + claims adjudication** — multi-party signed-document workflows; cert chain becomes audit trail.

Multi-chain via UOR-ADDR-1 adapters

Base ships first in this hackathon submission. The sibling implementation **AgentLevy-XRPL-UOR** targets XRPL XLS-100 SmartEscrow + RLUSD with **the same protocol primitives** — only the settlement adapter differs. **One protocol, two live chains**, proving UOR-ADDR-1's chain-binding adapter pattern works: any chain supporting a hashlock-conditional release can be added without changing the protocol layer (Hedera EVM, Solana, Sui — each gets an adapter; agents stay chain-agnostic).

Phase 4 (next): AgentCore Memory upgrade for stateful agents

Today's AWS Lambda sanctions agent is **stateless** — perfect for one-shot screening. Phase 4 upgrades it to a **stateful AgentCore-hosted variant** for cross-day fraud detection, learning patterns over time. The handler shape, request format, and orchestrator integration stay identical; only the runtime changes from Lambda to AgentCore Runtime + AgentCore Memory. **AgentCore makes the agent capable. UOR cert chains make its work verifiable. Together: regulated agent commerce, productized.**

Verifiable agent memory + AI inference provenance

The cert chain we ship for KYC is the same primitive used for **verifiable agent memory**. Every cert is a content-addressed, signed, anchored record of "this agent did this work on these inputs at this consensus-witnessed time." Stack many of these and you get an agent's complete, mathematically-verifiable history — the foundation for:

- **Long-horizon agent reputation** — not vendor-trusted scores, but a public-key-verifiable track record. An agent's past certs are its résumé.
- **AI inference provenance** — for enterprise AI governance: "show me, cryptographically, what model + version + prompt + inputs produced this output." Same `DerivationCert` shape; new operation types.
- **Memoization with audit** — when an agent re-uses a prior result instead of recomputing, the prior cert IS the citation. Cache hits become cryptographically auditable.
- **Cross-agent memory sharing** — an agent referencing another agent's prior work cites by content address, not by API. The reference resolves whether the original agent still exists or not. Composes naturally with MemWal-style memory protocols.

Phase 3: dNFT + SmartEscrow integration (XRPL-specific)

Layering XRPL's native **XLS-20 dynamic NFTs** + **XLS-100 SmartEscrow** on top of the AgentLevy cert chain produces three composable layers governing one workflow: cert chain (verifiable history) + dNFT (current state) + SmartEscrow (automated consequence). **The combination is uniquely cheap on XRPL** — the dNFT and SmartEscrow primitives are native, so AgentLevy layers on top without thousands of lines of custom contract code.

Flagship use case: AI model pay-per-inference with cryptographic enforcement. Model licensed as a dNFT; each inference produces a `DerivationCert`; cert submission updates the dNFT's usage counter; SmartEscrow releases per-inference payment + royalties to model creators automatically. Solves usage tracking, royalty enforcement, and inference provenance *simultaneously* — none of Coinbase x402, Virtuals ACP, or traditional API-key billing solve all three. **TAM: the full AI inference economy** (projected hundreds of billions by late 2020s, currently vendor-trusted billing with weak provenance).

Other dNFT-enabled markets the same pattern unlocks: **portable KYC attestations** (verified KYC travels with the customer across institutions), **tranchéd M&A escrow** (programmable milestone-based release), **title NFTs** (each conveyance is a state transition; closing escrow auto-releases on `NEW_OWNER`), **patient consent NFTs** (HIPAA-friendly; revocable), **carbon credit verification** (solves voluntary carbon market's double-counting), **SLA-enforced subscriptions** (auto-refunds on breach), **supply chain provenance**.

Productization → Kessai (next slide)

The open-source reference protocol is AgentLevy. The commercial layer is Kessai — visualizer UI, enterprise SDKs, regulatory-evidence packs, channel licensing. **Same protocol, productized.**

SLIDE 13 — What this becomes



KESSAI

決 済

TRUST, COMPUTED.

AgentLevy is the open reference implementation. Kessai is the productization.

- **AgentLevy** — open-source, CC0/Apache 2.0, MIT-vendored. The protocol other people can build on. Standards-aligned. Auditable. Forkable.
- **Kessai** — the commercial layer: enterprise SaaS, KYC + AML + M&A escrow visualizer, regulatory-evidence packs, channel-licensed to compliance vendors.

The protocol stays open. The standards stay free. The product is what makes verifiable settlement turnkey for enterprises.

Try it · Read the code · Help shape the standards:

-  GitHub: github.com/maurathat/AgentLevy-Base-UOR
-  Standards: VTEAI ERC draft + UOR-ADDR-1 community proposal
-  Pitch on file: ask any judge or booth visitor; we love a hard question
-  Hiring: 2 senior engineers (Python SDK + multi-chain settlement: Base, Hedera, XRPL) — pre-seed open

— Maura Clark · maurathat — [contact info]

AgentLevy v2 (Base × PRISM × Hedera × AWS × KIRO). Brand visuals + ecosystem logos hosted at github.com/maurathat/kessai-pitch-assets. Apache License 2.0.